Computer Science & IT

Theory of Computation



CFL

Lecture No. 9



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

Ambi grammar, Ambiguous language

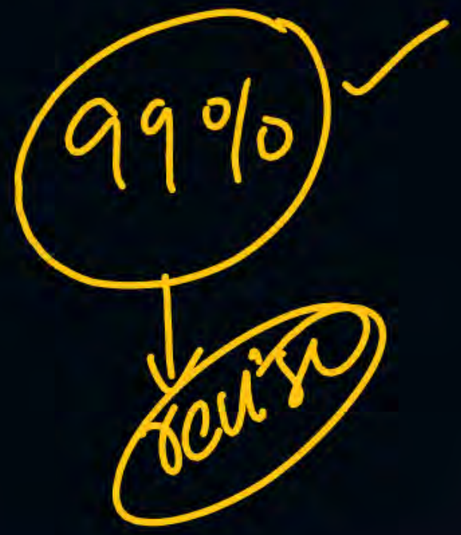
Topics to be Covered



- Topic
- Topic
- Topic

amb', LL(k) LR(k)
DCFL

→ If a grammar 'G' is ambous for an unambiguous language, Can we give an unambiguous grammar? (True/False)



RL → FA

DCFL → DPDA

CFL → PDA

CSL → Linear bounded automata

Rec → Halting TM

RE → TM

Turing machine

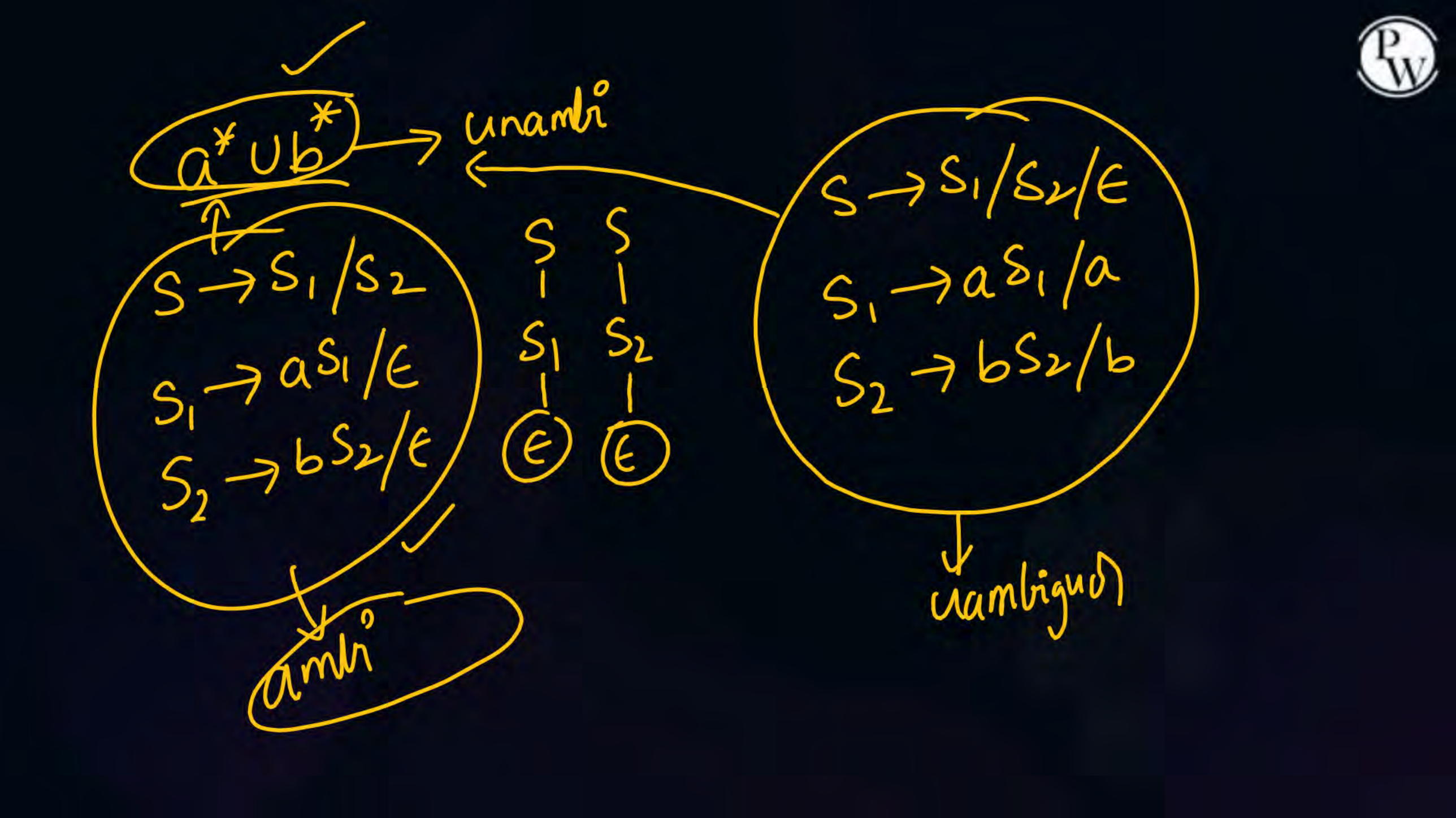
→ Certain types of languages cannot be ambiguous

Ex: Reg, DCFL are unambiguous



unambiguous languages

ambiguity in languages starts beyond DCFL


a* U b*
unambi
S → S1 / S2
S1 → a S1 / ∈
S2 → b S2 / ∈
S S
S1 S2
∈ ∈
S → S1 / S2 / ∈
S1 → a S1 / a
S2 → b S2 / b
ambi
unambiguous

There may be some ambiguous grammars for RL and DCFL. But, RL and DCFL are unambiguous languages

Reg lang

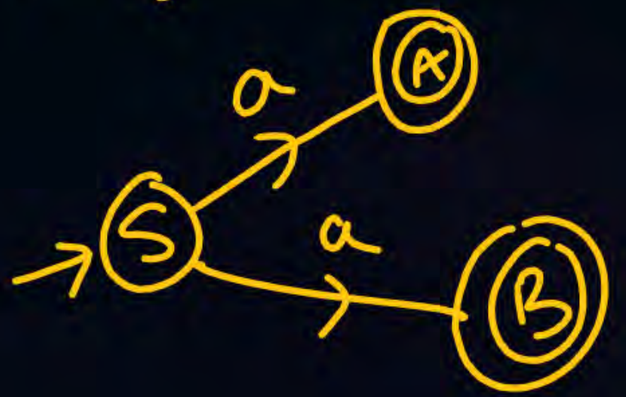
Reg lang → (min) DFA → grammar

↓

unambi because there is no choice



A grammar from an NFA may or may not be unambiguous

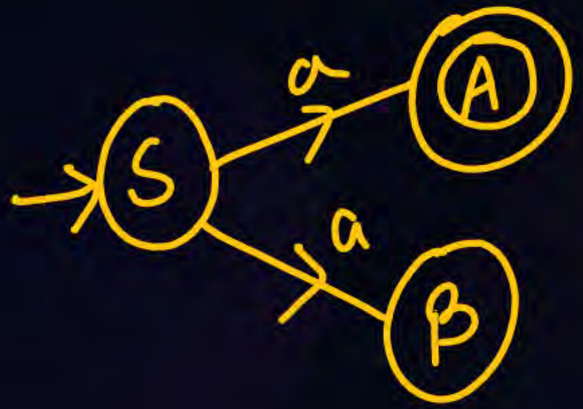


$S \to aA / aB$

$A \to \epsilon$

$B \to \epsilon$





$S \to aA / aB$

$A \to \epsilon$

unambiguous

NFA → grammar → may / may not be ambiguous

DFA → grammar → always unambiguous

DPDA → grammar → always unambiguous

↓

no choice



→ amb & unamb

→ alway unamb

For $\bar{RL}$ and $\bar{DCFL}$, ∃ atleast one unambiguos grammar

There are some non DCFL languages which are unambiguous

$L = \{a$

NOT DCFL

$L = \{a^n b^n\} \cup \{a^n b^{2n}\} / n \geq 0$

$S \to S_1 / S_2 / \epsilon$

$S_1 \to a S_1 b / \epsilon$

$S_2 \to a S_2 bb / \epsilon$

ambi

S S

| |

S_1 S_2

| |

ϵ ϵ

$S \to S_1 / S_2 / \epsilon$

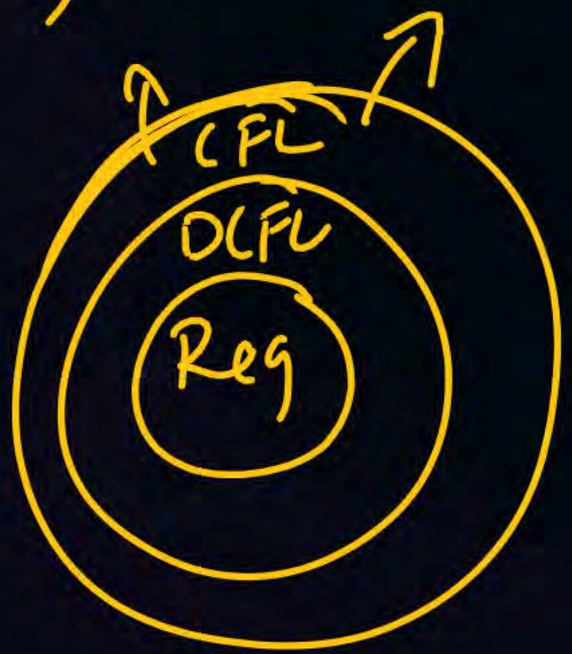
$S_1 \to a S_1 b / ab$

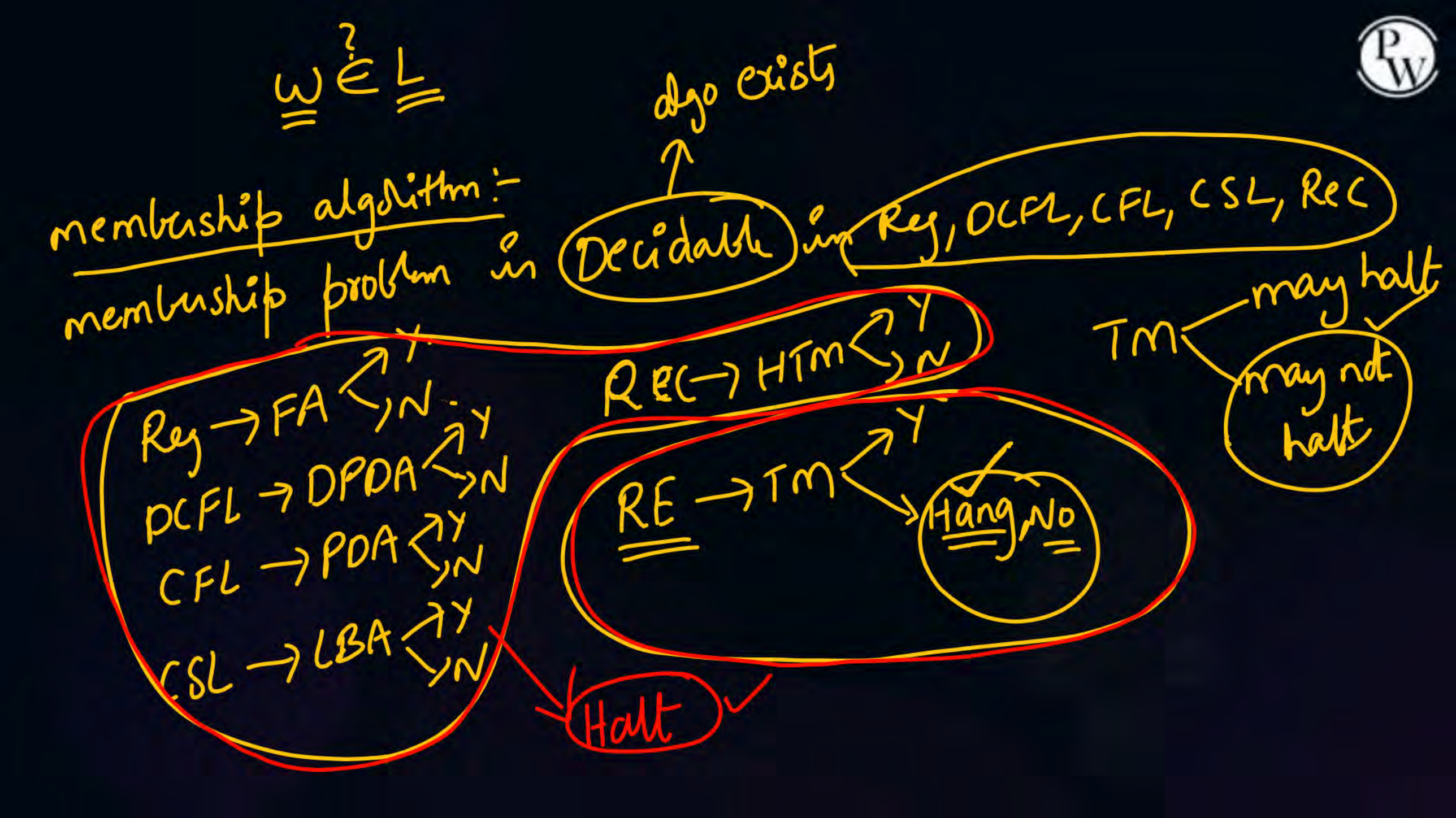
$S_2 \to a S_2 bb / abb$

unambiguous grammar

RL, DCFL → unamb°

Beyond DCFL → amb & unamb°




$w \overset{?}{\in} L$
membership algorithm :-
membership problem is
Decidable
algo exists
Reg, DCFL, CFL, CSL, REC
Reg → FA Y/N
DCFL → DPDA Y/N
CFL → PDA Y/N
CSL → LBA Y/N
REC → HTM Y/N
RE → TM Y
Hang, No
Halt
TM
may halt
may not halt

Closure, Decidable

↓ ↓

Table Table

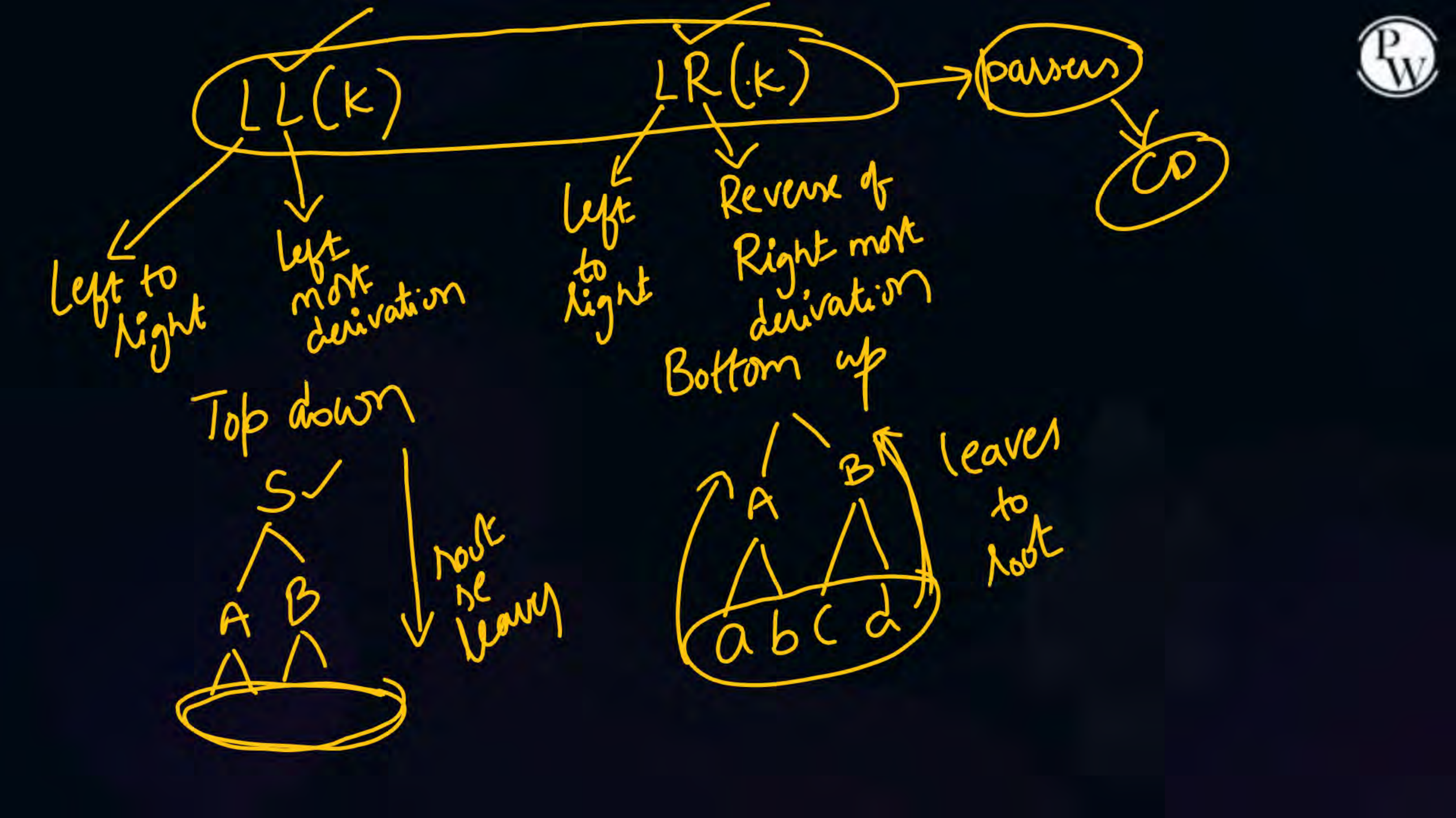
membership algorithms for CFL

1) Brute force → exponential time

2) CYK → $O(n^3)$

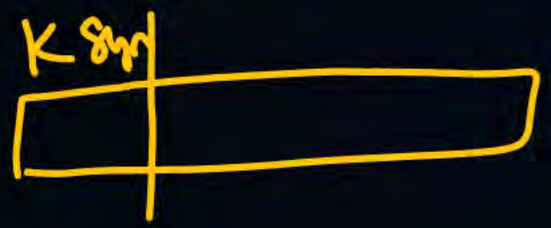
3) LL(k), LR(k) → $O(n)$ ✓

Compilers ✓


LL(k)
LR(k)
parsers
CD
Left to right
Left most derivation
Left to right
Reverse of Right most derivation
Top down
S
A
B
root se leaves
Bottom up
A
B
leaves to root
a b c d

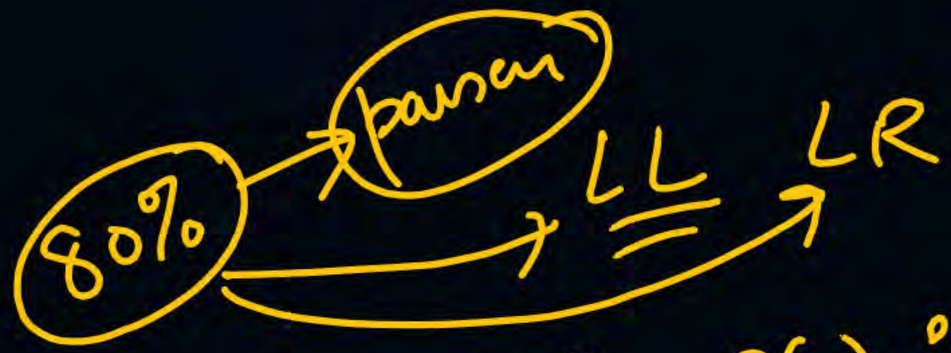
Properties of LL(k) and LR(k) :-(

1) LL(k) and LR(k) grammars are unambiguous.
   ∴ Suitable for compiler design
2) Some unambiguous grammars are neither LL(k) nor LR(k)
3) If a grammar is ambiguous, then Surely it is not LL(k) or LR(k)



CD

LL(K) :- A grammar is LL(k), when k symbols of i/p are presented to the compiler at a time, the production that is used in every step of derivation is uniquely determined



CD

LR(k) :- A grammar G is LR(k) iff when k i/p symbols are presented to compiler at a time, the production that is used in every step of derivation is determined uniquely.

$\forall$ DCFL $\longrightarrow$ $\exists$ LR(k)



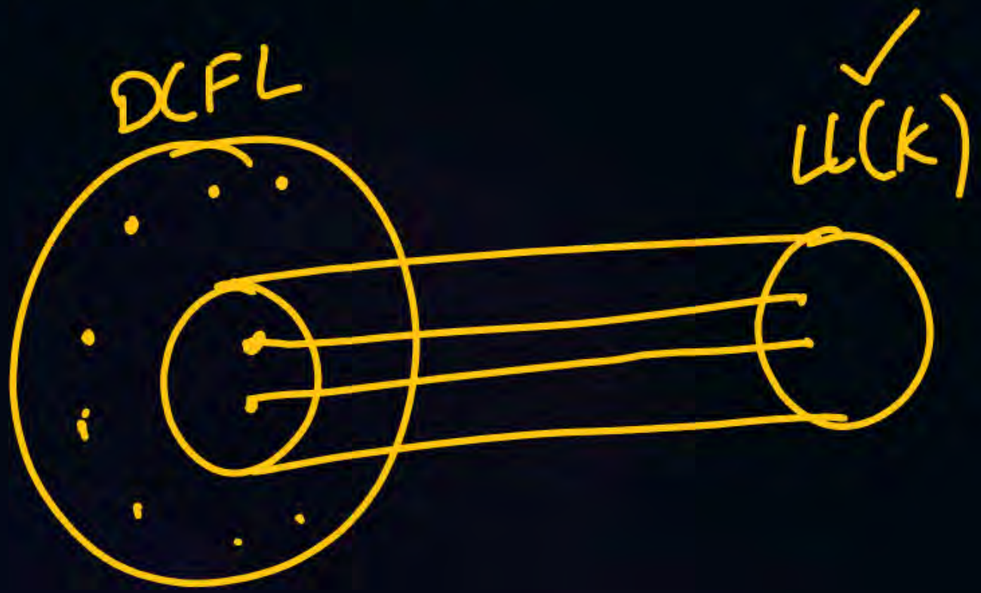
For every DCFL there exists an LR(k) grammar

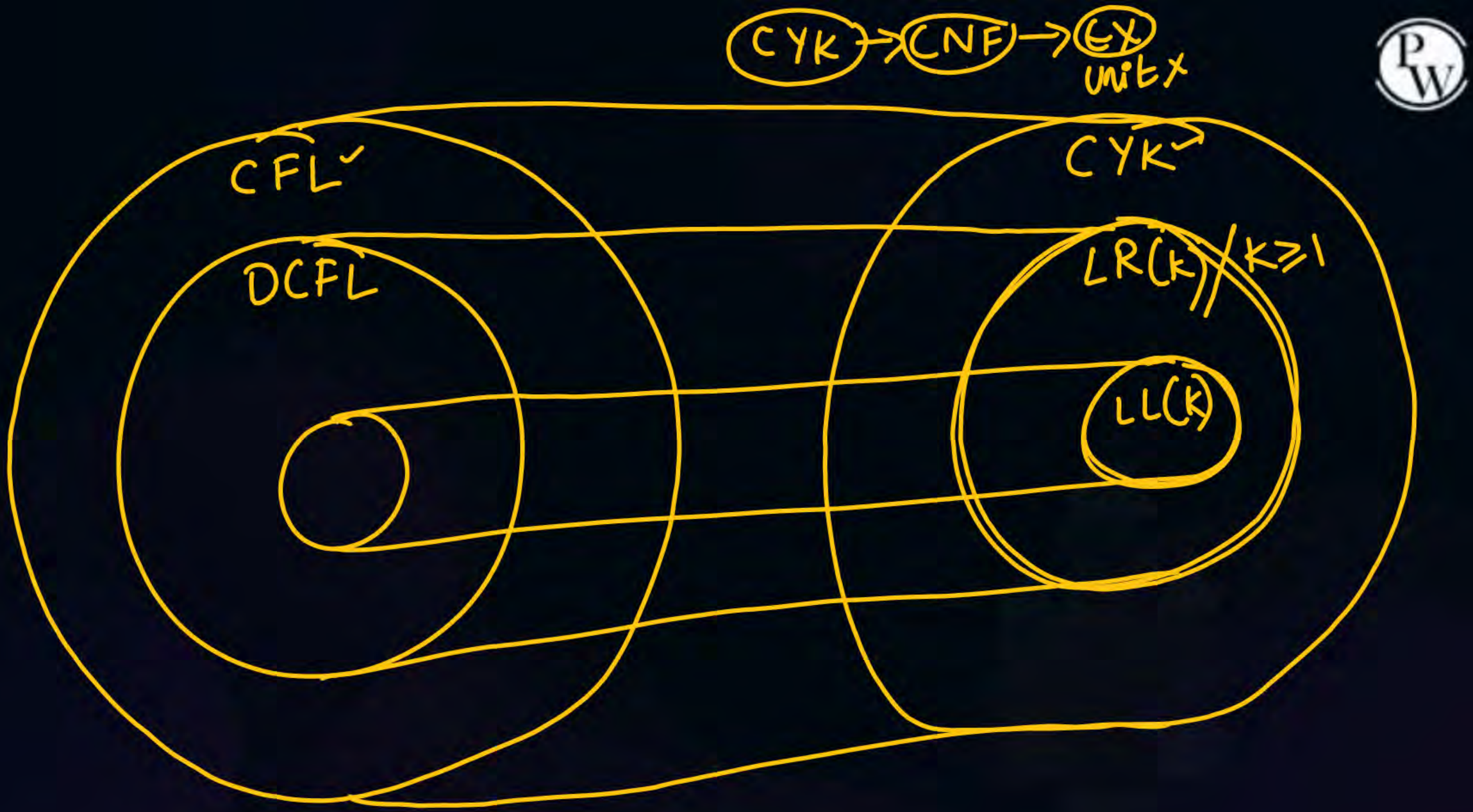
$\forall$ LR(k) $\longrightarrow$ $\exists$ atleast one DCFL



$\forall$ LL(k) $\longrightarrow$ $\exists$ atleast on DCFL

But all DCFL's need not have LL(k) grammar

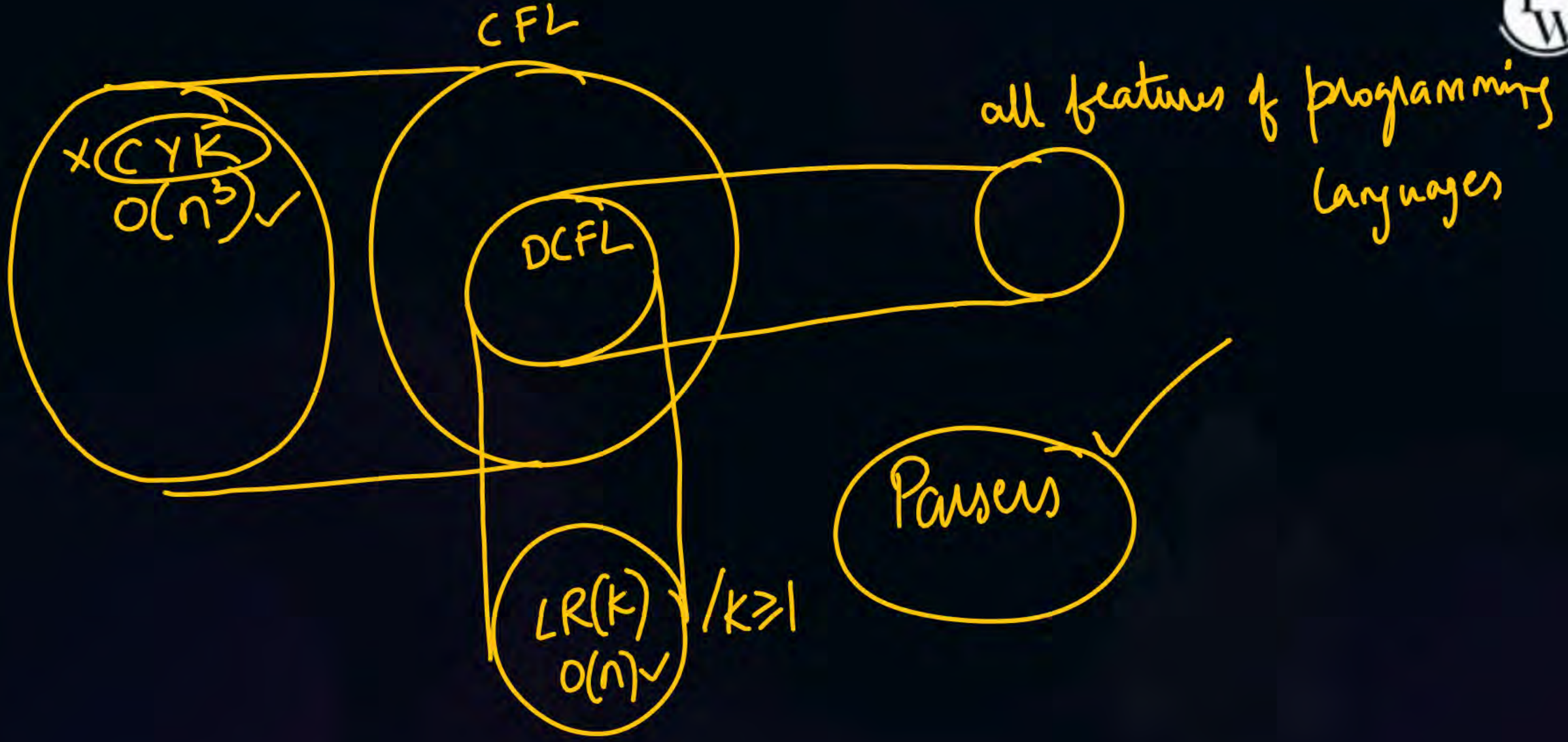



CYK → CNF → ε unit ×
CFL ✓
DCFL
CYK →
LR(k)
K≥1
LL(k)

→ In LL(k) and LR(k) every string will have exactly one derivation tree. Because LL(k) and LR(k) are unambiguous

→ LL(k) and LR(k) can parse in O(n) time

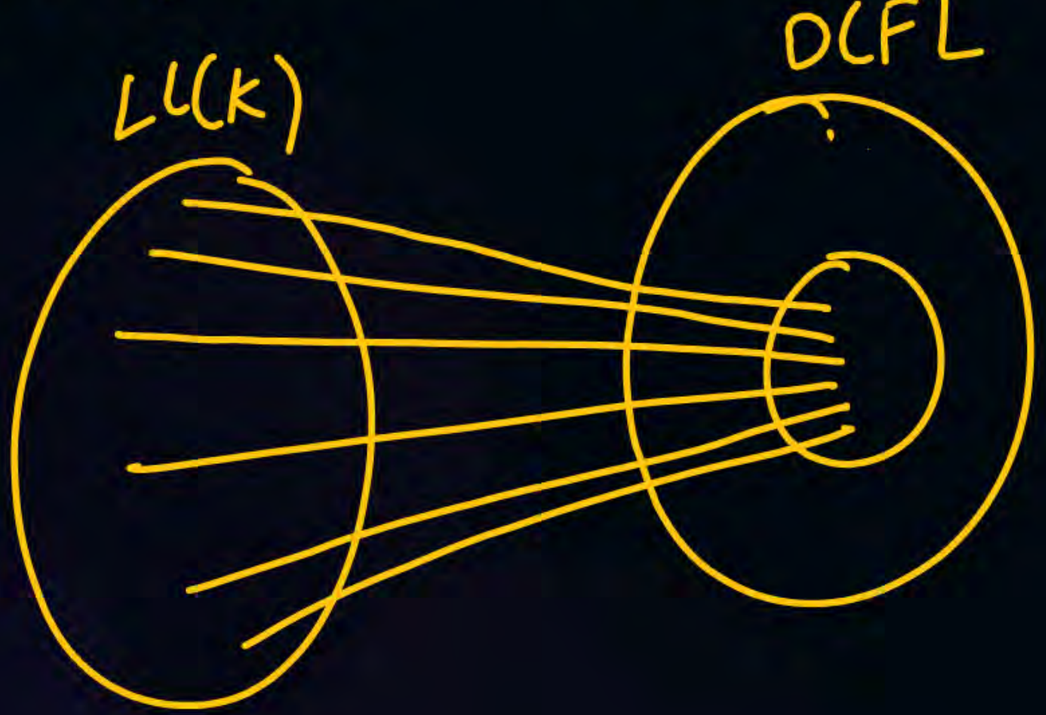
→ Every feature of any programming language is in DCFL. So we dont use CYK in compiler


CFL
CYK
O(n^3)
DCFL
all features of programming languages
Parsers
LR(k)
O(n)
/k≥1

Every DCFL has a corresponding LR(k)



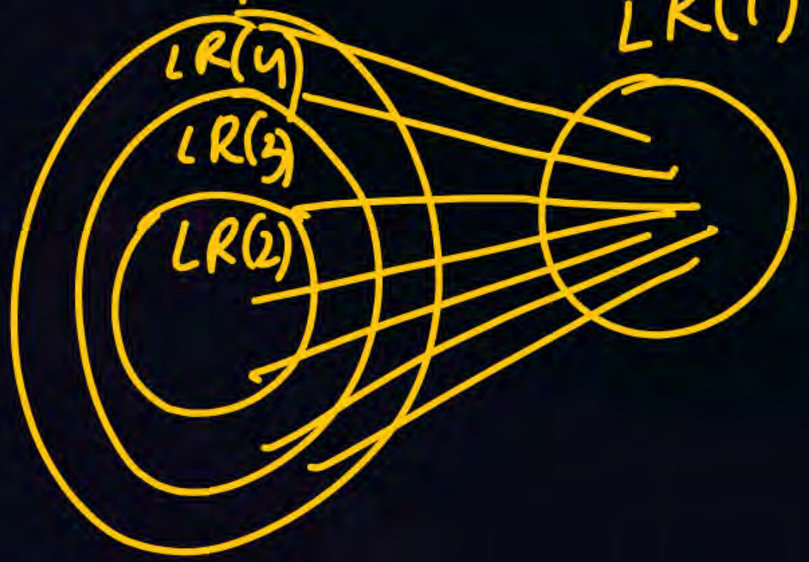
Every LL(k) is DCFL



→ ∃ an algo to check if a grammar is LL(k) (decidable)

→ ∃ " " " " LR(k) (decidable)

→ if G is LR(k) / k≥2 then ∃ G' which is LR(1) for the same grammar



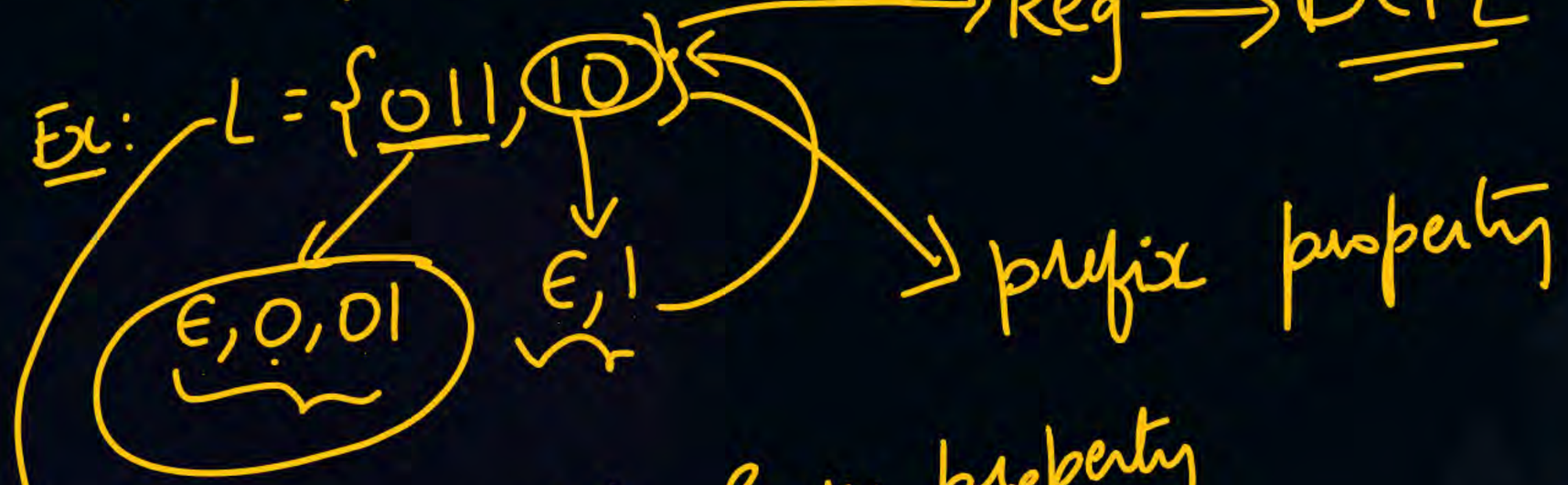
So all we need is LR(1)

$\forall$ DCFL with prefix property, we can generate LR(0)



For every DCFL with prefix property there will be a LR(0) grammar

DCFL 'L' has a prefix property if $\forall w \in L$, there is no proper prefix of $w \in L$



↳ DCFL with Prefix property

So there exists a LR(0) grammar for this language

$L = \{0\,11,\ 10,\ 1\} \rightarrow$ Reg $\rightarrow$ DCFL

$\epsilon, 0, 01 \quad \epsilon, 1, 1 \rightarrow$ no prefix property

For this language there is no LR(0) glamman

$L = (a^* b^*) \longrightarrow$ Reg $\longrightarrow$ DCFL

$L = \{\epsilon, a, aa \cdots, b, bb \cdots, ab, aab, abb \cdots\}$



whenever ϵ is present in langyge then it does not have prefix property

This lanyge doesn't have LR(0) grammar

$\{a^n b^n / n \geq 0\} \longrightarrow$ DCFL

ϵ is in the language, so no prefix property

$\{a^n b^n / n \geq 1\} \longrightarrow$ has prefix property $\neq$ DCFL

$\downarrow$

LR(0) grammar exists

aaabbb

$\downarrow$ proper prefixes

ϵ ✓, a ✓, aa ✗, aaa ✗, aaab ✗, aaabb ✗

$L = \{\epsilon\} \longrightarrow$ has no prefix property

$\epsilon \cdot \epsilon$

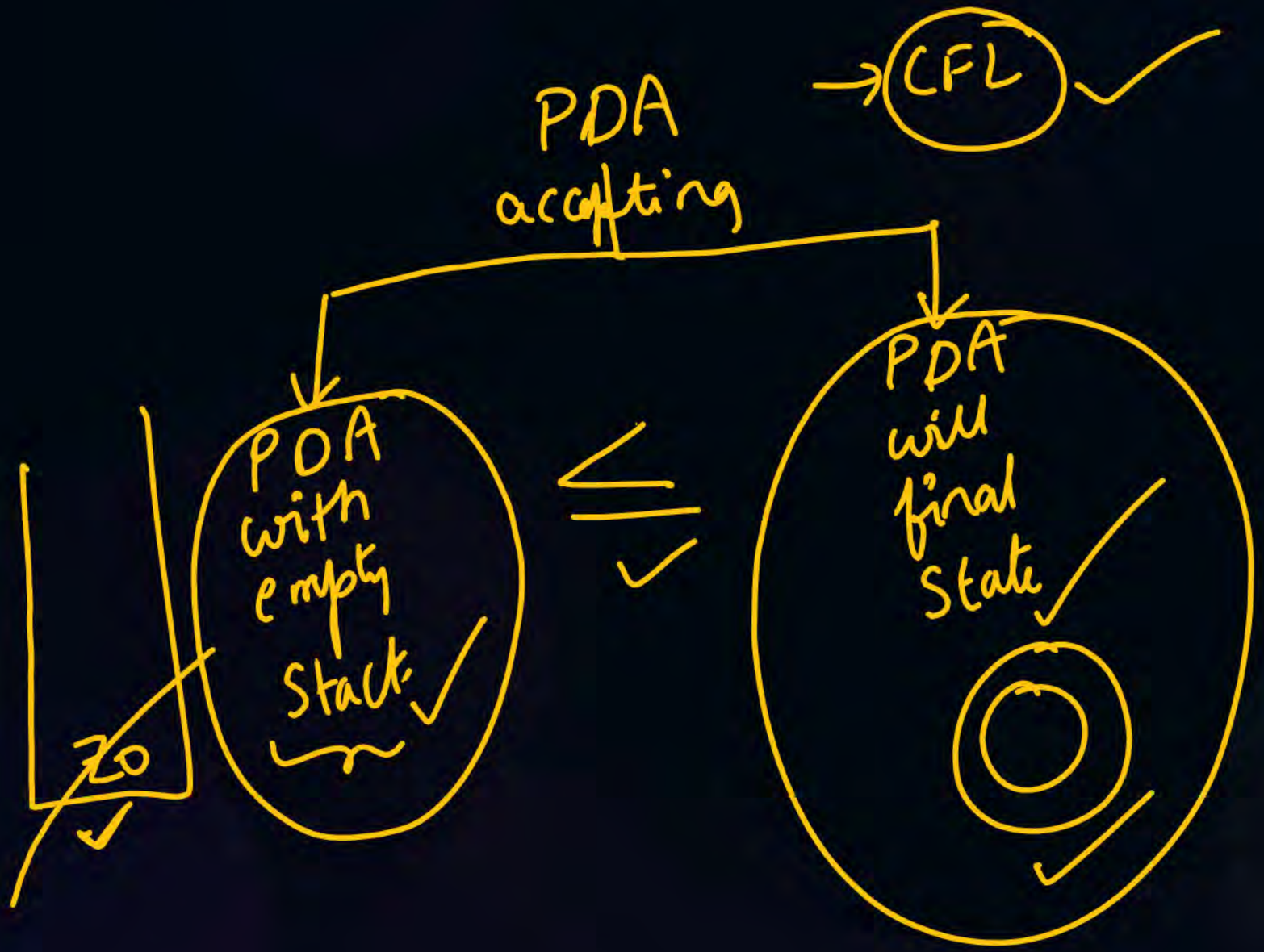
$a^+b^{(*)^0}$ → Reg → (DCFL)

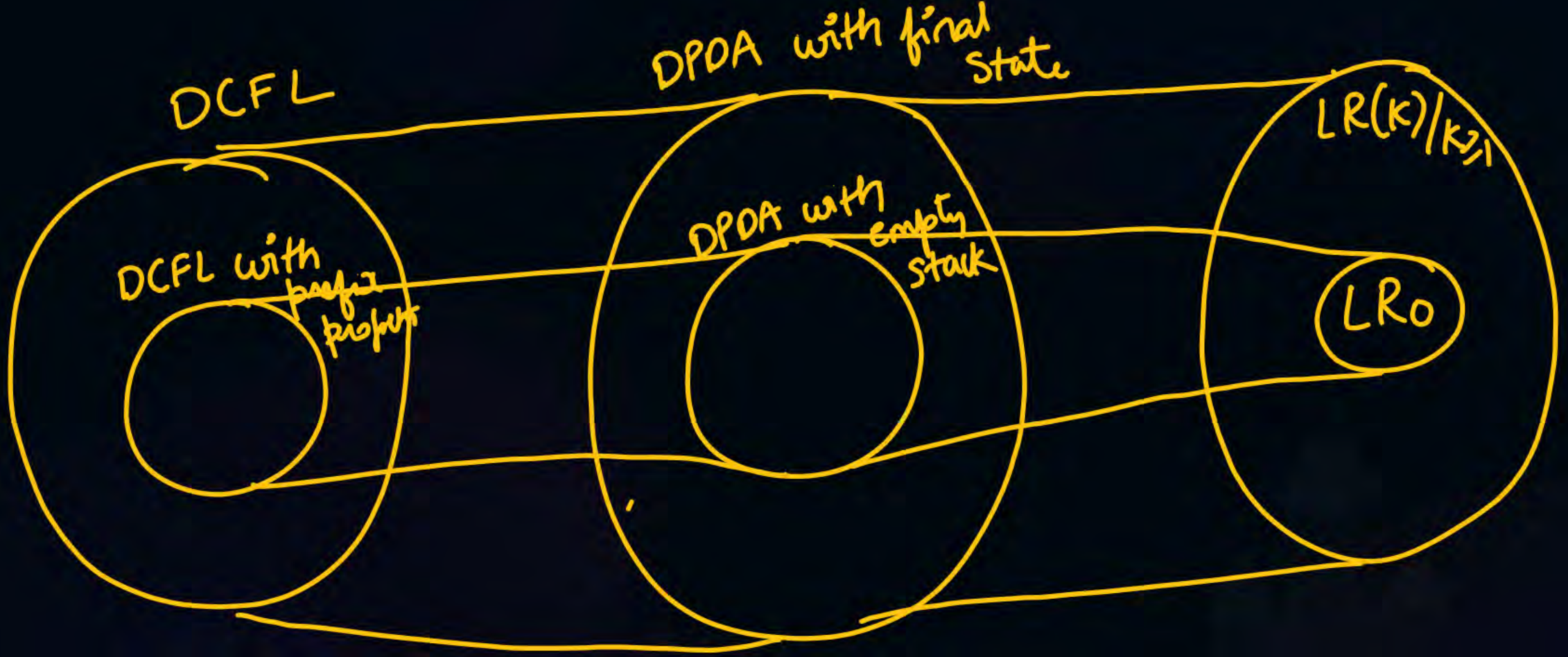
(a), aa, aaa ----; ab, aab, abb ---



→ has (no prefix property)

NO LR(0) grammar exists


PDA accepting
CFL
PDA with empty Stack
PDA will final State
Z0


DCFL
DPDA with final state
LR(k)/k≥1
DCFL with prefix property
DPDA with empty stack
LR0

Algorithm in CFL :-

1) a) Removal of ϵ productions
   b) " " unit "
   c) " " useless variables
   d) Removal of left recursion
   e) Removal of left factoring

2) Conversions
   a) CFG → CNF
   b) CFG → GNF

all these problems are decidable

→ ϵ and unit productions have to be removed before a grammar is converted to CNF or GNF

→ If the language contains 'ϵ', then all the ϵ productions cannot be eliminated. So CNF & GNF are not possible.

→ ∀ CFG which are ∈-free, ∃ CNF

→ " " " ∃ GNF

→ (T/F) For every CFL, there is a CNF (false)

→ (T/F) for " " GNF (false)

Order :-

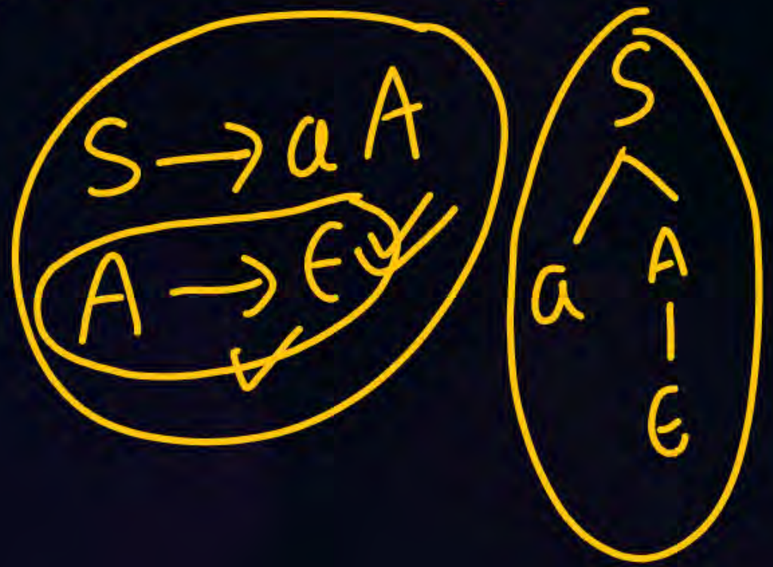
a) Remove $\in$ production ✓

b) Remove unit productions ✓

c) Remove useless variables ✓



Presence of ϵ production in a grammar doesn't mean 'ϵ' is in the language



$L = \{a\}$

If ϵ is present in the language then there will be atleast one ϵ production in the grammar

$S \rightarrow aAB/aB/B$

$B \rightarrow ab/Ab/bA$

$A \rightarrow \epsilon$

Nullable $\rightarrow \{A, S\}$ ✓

$S \rightarrow$ nullable

$\Downarrow$

ϵ is not in 'L'

$S \rightarrow aAB/aA/\underline{B}$

$B \rightarrow ab/Ab/\underline{A}$

$\underline{A} \rightarrow \epsilon$

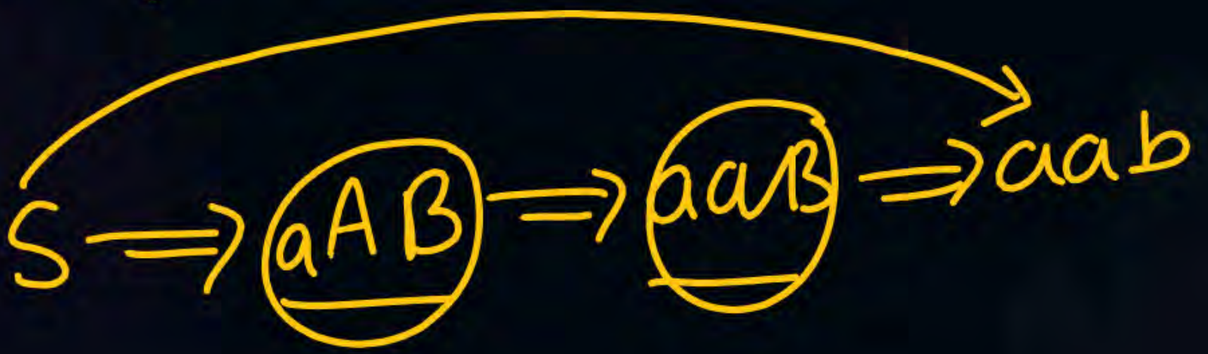
$nullable = \{\underline{A}, \underline{B}, \underline{S}\}$

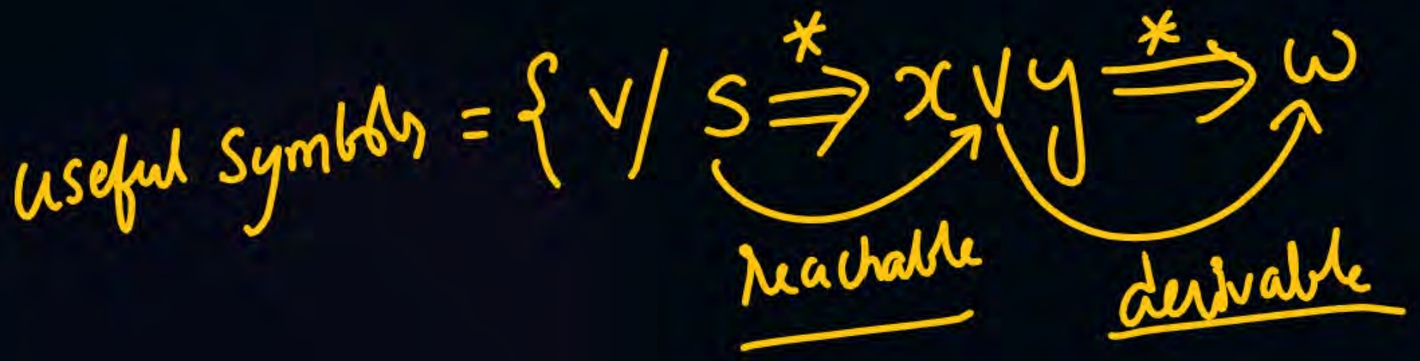
ϵ is in language

S - B - A - ϵ

NO CNF
No GNF

A Varible V is useless if it doesnt appear in any Sentential form




useful symbols = { v / S ⇒* x v y ⇒* w
reachable
derivable

→ If a language is inherently ambiguous, the removal of LR and LF doesn't give unambiguous

Removing left recursion and left factoring :-

LR and LF (may) lead to ambiguity

(T/F) A grammar has no left recursion and no left factoring

What is the language?

a) ambiguous b) unambiguous c) Can't say



$S \to S_1 / S_2$

$S_1 \to a S_1 / \epsilon$

$S_2 \to b S_2 / \epsilon$

THANK - YOU